

Python — Fiche de révision

Concours ANP — Cadre Data Scientist

1 Structures de données de base

1.1 List

Une **list** est une séquence **ordonnée**, **modifiable** (*mutable*), qui peut contenir des doublons.

```
ports = ["Casablanca", "Tanger", "Agadir"]
ports.append("Nador") # ajout en fin
ports[0] # "Casablanca" (accès par index)
ports[0] = "Mohammedia" # modification possible (mutable)
len(ports) # 4
```

- Notation []. Indexable, itérable, ordonnée.
- Usage typique : collection d'éléments qui peut évoluer (ajout/suppression/tri).

1.2 Tuple

Un **tuple** est une séquence ordonnée mais **immuable** (*immutable*) : impossible de la modifier après création.

```
coord = (33.5731, -7.5898) # latitude, longitude
coord[0] # 33.5731
# coord[0] = 34.0 # ERREUR : TypeError, tuple non modifiable
```

- Notation (). Plus rapide et plus sûr qu'une list quand les données ne doivent pas changer.
- Usage typique : coordonnées, clé de dictionnaire (car *hashable*), retour multiple d'une fonction.

1.3 Dict

Un **dict** (dictionnaire) stocke des paires **clé-valeur**, non ordonnées avant Python 3.7 (ordre d'insertion préservé depuis).

```
navire = {"nom": "MSC Zoe", "tirant_eau": 14.5, "conteneurs": 19000}
navire["nom"] # "MSC Zoe"
navire["statut"] = "a quai" # ajout d'une clé
navire.get("vitesse", 0) # 0 (valeur par défaut si absente)
navire.keys() # dict_keys(['nom', 'tirant_eau', ...])
```

- Notation {clé: valeur}. Accès en $O(1)$ par clé (table de hachage).
- Usage typique : représenter un enregistrement structuré (équivalent d'une ligne de table).

1.4 Set

Un **set** est une collection **non ordonnée** d'éléments **uniques** (pas de doublons).

```
types_navires = {"vraquier", "porte-conteneurs", "petrolier", "vraquier"}
# {"vraquier", "porte-conteneurs", "petrolier"} -> doublon supprime
types_navires.add("roulier")
"vraquier" in types_navires # True, test d'appartenance en  $O(1)$ 
```

- Notation {}. Idéal pour dédupliquer une liste ou tester une appartenance rapidement.
- Supporte les opérations ensemblistes : union (|), intersection (&), différence (-).

1.5 Tableau récapitulatif

Type	Ordonné	Modifiable	Doublons	Notation
list	Oui	Oui	Oui	[1, 2, 3]
tuple	Oui	Non	Oui	(1, 2, 3)
dict	Oui (3.7+)	Oui	Clés uniques	{"a": 1}
set	Non	Oui	Non	{1, 2, 3}

2 Compréhensions (comprehension)

Une **compréhension** construit une nouvelle collection en une seule ligne, de façon plus lisible et souvent plus rapide qu'une boucle **for** classique.

```
# List comprehension
carres = [x**2 for x in range(10)]
pairs = [x for x in range(20) if x % 2 == 0]

# Dict comprehension
noms_longueurs = {nom: len(nom) for nom in ["Tanger", "Agadir", "Nador"]}

# Set comprehension
lettres_uniques = {c for c in "portuaire"}
```

- Syntaxe générale : [expression for élément in itérable if condition].
- Préférée aux boucles **for** explicites pour des transformations simples (plus *pythonique*).

3 Fonctions

```
def taux_occupation(conteneurs_charges, capacite_max):
    """Retourne le taux d'occupation d'un terminal en pourcentage."""
    if capacite_max == 0:
        raise ValueError("La capacite ne peut pas etre nulle")
    return (conteneurs_charges / capacite_max) * 100

# Argument par default
def alerte(taux, seuil=90):
    return taux >= seuil

# *args et **kwargs : nombre variable d'arguments
def total(*valeurs, **options):
    return sum(valeurs)

total(10, 20, 30) # 60

# Fonction lambda (anonyme, une seule expression)
carre = lambda x: x ** 2
carre(5) # 25
```

- ***args** : arguments positionnels variables (regroupés en tuple).
- ****kwargs** : arguments nommés variables (regroupés en dict).
- Une fonction sans **return** explicite renvoie **None**.

4 Exceptions

La gestion des exceptions permet de réagir aux erreurs à l'exécution sans faire planter le programme.

```

try:
    taux = conteneurs / capacite
except ZeroDivisionError:
    print("Capacite nulle, impossible de calculer le taux")
except (TypeError, ValueError) as e:
    print(f"Donnee invalide : {e}")
else:
    print("Calcul reussi") # execute seulement si pas d'erreur
finally:
    print("Traitement termine") # toujours execute (nettoyage, fermeture fichier...)

# Lever une exception explicitement
if capacite < 0:
    raise ValueError("La capacite ne peut pas etre negative")

```

- **Jamais de except: nu** (bare except) : attraper toutes les erreurs sans distinction masque les vrais bugs — cible toujours une exception précise.
- **else** : code exécuté seulement si le bloc **try** n'a levé aucune exception.
- **finally** : code exécuté dans tous les cas (erreur ou non) — utilisé pour libérer des ressources (fichier, connexion DB).
- Exceptions courantes : `ValueError`, `TypeError`, `KeyError`, `IndexError`, `ZeroDivisionError`, `FileNotFoundError`.

5 OOP basique (Programmation Orientée Objet)

```

class Navire:
    # Attribut de classe (partage par toutes les instances)
    compteur = 0

    def __init__(self, nom, tirant_eau):
        self.nom = nom # attribut d'instance
        self.tirant_eau = tirant_eau
        Navire.compteur += 1

    def peut_accoster(self, profondeur_quai):
        """Methode d'instance : logique métier propre a l'objet."""
        return self.tirant_eau <= profondeur_quai

    def __str__(self):
        return f"Navire({self.nom}, tirant_eau={self.tirant_eau}m)"

class PorteConteneurs(Navire): # heritage
    def __init__(self, nom, tirant_eau, capacite_evp):
        super().__init__(nom, tirant_eau) # appel au constructeur parent
        self.capacite_evp = capacite_evp

    def peut_accoster(self, profondeur_quai): # redefinition (polymorphisme)
        return super().peut_accoster(profondeur_quai) and self.capacite_evp < 20000

navire = PorteConteneurs("MSC Zoe", 14.5, 19224)
navire.peut_accoster(15) # True
print(navire) # Navire(MSC Zoe, tirant_eau=14.5m)

```

5.1 Concepts clés

- **Classe** : modèle/plan (*blueprint*) définissant attributs et méthodes. **Objet (instance)** : réalisation concrète d'une classe.
- **self** : référence à l'instance courante, premier paramètre de toute méthode d'instance.
- **__init__** : constructeur, appelé automatiquement à la création de l'objet.
- **Encapsulation** : regrouper données et comportements dans une même entité ; convention `_attribut` (protégé) ou `__attribut` (privé).
- **Héritage** : une classe fille (*PorteConteneurs*) réutilise et étend le comportement d'une classe mère (*Navire*) via `super()`.
- **Polymorphisme** : une méthode redéfinie dans la classe fille (*peut_accoster*) remplace celle du parent tout en gardant la même interface.

6 Questions types (révision active)

1. Quelle est la différence entre une list et un tuple ? Pourquoi utiliser un tuple plutôt qu'une list ?
2. Pourquoi un dict est-il rapide pour rechercher une valeur par clé ?
3. À quoi sert un set, et quelle opération illustre le mieux son intérêt ?
4. Réécrivez une boucle `for` qui construit une liste des carrés pairs en une list comprehension.
5. Quelle est la différence entre `*args` et `**kwargs` ?
6. Pourquoi ne faut-il jamais utiliser un `except:` nu ?
7. Quelle est la différence entre `try/except/else` et `try/except/finally` ?
8. Qu'est-ce que `self` dans une méthode de classe Python ?
9. Expliquez l'héritage et le polymorphisme avec un exemple simple.

7 Corrigé des questions types

Q1. Différence list / tuple, pourquoi utiliser un tuple ?

Les deux sont des séquences ordonnées pouvant contenir des doublons. La list est **modifiable** (on peut ajouter, retirer, réassigner des éléments) ; le tuple est **immuable** (une fois créé, son contenu ne change plus). On préfère un tuple quand les données ne doivent pas être modifiées par erreur (coordonnées, constantes), quand on veut l'utiliser comme clé de dictionnaire (il est *hashable*, contrairement à la list), ou pour de meilleures performances sur des données fixes.

Q2. Pourquoi un dict est-il rapide pour rechercher une valeur par clé ?

Un dict est implémenté avec une **table de hachage** : chaque clé est transformée par une fonction de hachage en une position mémoire, ce qui permet d'accéder directement à la valeur sans parcourir toute la structure. La recherche, l'insertion et la suppression sont en moyenne en $O(1)$, contre $O(n)$ pour chercher un élément dans une list (parcours séquentiel).

Q3. À quoi sert un set, quelle opération illustre le mieux son intérêt ?

Un set sert à stocker des éléments **uniques** et à tester une appartenance très rapidement ($O(1)$ comme le dict, car basé sur le même principe de hachage). L'opération qui illustre le mieux son intérêt est la **déduplication** : transformer une list contenant des doublons en set supprime automatiquement les répétitions (`set(ma_liste)`). Les opérations ensemblistes (union, intersection, différence) sont aussi typiques, par exemple pour comparer deux populations de navires.

Q4. Boucle for vers list comprehension (carrés pairs) :

```
# Version boucle for
resultat = []
for x in range(20):
    if x % 2 == 0:
        resultat.append(x**2)

# Version list comprehension (equivalente, plus concise)
resultat = [x**2 for x in range(20) if x % 2 == 0]
```

Q5. Différence `*args` / `**kwargs` ?

`*args` regroupe un nombre variable d'arguments **positionnels** dans un **tuple** à l'intérieur de la fonction. `**kwargs` regroupe un nombre variable d'arguments **nommés** (`cle=valeur`) dans un **dict**. Les deux permettent d'écrire des fonctions flexibles qui acceptent un nombre inconnu d'arguments à l'avance.

Q6. Pourquoi ne jamais utiliser un `except : nu` ?

Un `except : nu` attrape **toutes** les exceptions sans distinction, y compris des erreurs de programmation graves (`KeyboardInterrupt`, `SystemExit`, `MemoryError`) ou des bugs inattendus qu'on aurait dû voir et corriger. Cela masque la vraie cause d'un problème et rend le débogage très difficile. Il faut toujours cibler une exception précise (`except ValueError :`) pour ne gérer que le cas d'erreur réellement anticipé.

Q7. Différence `try/except/else` et `try/except/finally` ?

`else` contient du code exécuté **uniquement si aucune exception** n'a été levée dans le bloc `try` — utile pour séparer la logique "tout s'est bien passé" de la logique de traitement d'erreur. `finally` contient du code exécuté **dans tous les cas**, qu'il y ait eu une exception ou non (et même si un `return` a eu lieu) — utilisé typiquement pour libérer des ressources (fermer un fichier, une connexion à la base de données).

Q8. Qu'est-ce que `self` ?

`self` est la référence à l'**instance courante** de l'objet sur laquelle une méthode est appelée. C'est le premier paramètre de toute méthode d'instance en Python (bien qu'il ne soit jamais passé explicitement lors de l'appel : `navire.peut_accoster(15)` envoie automatiquement `navire` comme `self`). Il permet à la méthode d'accéder et de modifier les attributs propres à cet objet précis.

Q9. Héritage et polymorphisme, exemple simple ?

L'**héritage** permet à une classe fille de réutiliser les attributs et méthodes d'une classe mère sans les réécrire, via `class Fille(Mere) :` et `super()` pour appeler explicitement le comportement du parent. Dans l'exemple, `PorteConteneurs` hérite de `Navire` et récupère `nom`, `tirant_eau` et `compteur` sans les redéfinir. Le **polymorphisme** permet à la classe fille de **redéfinir** une méthode héritée (`peut_accoster`) en gardant la même signature mais un comportement différent (ici, on ajoute une condition sur la capacité) : le code appelant utilise la même interface (`navire.peut_accoster(...)`) sans se soucier du type réel de l'objet.